

OpenBMC User's Guide

October 20, 2025

1 OpenBMC REST API

The OpenBMC REST API is disabled by default in bmcweb. Most of the functionality is available on Redfish. This document provides some basic structure and usage examples for the OpenBMC REST interface.

The schema for the rest interface is directly defined by the OpenBMC D-Bus structure. Therefore, the objects, attributes and methods closely map to those in the D-Bus schema.

For a quick explanation of HTTP verbs and how they relate to a RESTful API, see <http://www.restapitutorial.com/lessons/httpmethods.html>.

1.1 Authentication

See the details on authentication at [REST-cheatsheet](#).

This tutorial uses token based authentication method:

```
export bmc=xx.xx.xx.xx
export token=`curl -k -H "Content-Type: application/json" -X POST https://${bmc}/login -d '{"username": "root", "password": "root"}'`
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/...
```

1.2 HTTP GET operations & URL structure

There are a few conventions on the URL structure of the OpenBMC rest interface. They are:

- To query the attributes of an object, perform a GET request on the object name, with no trailing slash. For example:

```
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/inventory/system
{
  "data": {
    "AssetTag": "",
    "BuildDate": "",
    "Cached": true,
    "FieldReplaceable": true,
    "Manufacturer": "",
    "Model": "",
    "PartNumber": "",
    "Present": true,
    "PrettyName": "",
    "SerialNumber": ""
  },
}
```

```

    "message": "200 OK",
    "status": "ok"
}

```

- To query a single attribute, use the `attr/<name>` path. Using the system object from above, we can query just the Name value:

```

curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/inventory/system/attr/Cached
{
  "data": true,
  "message": "200 OK",
  "status": "ok"
}

```

- When a path has a trailing-slash, the response will list the sub objects of the URL. For example, using the same object path as above, but adding a slash:

```

curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/
{
  "data": [
    "/xyz/openbmc_project/Chassis",
    "/xyz/openbmc_project/Hiomapd",
    "/xyz/openbmc_project/Ipmi",
    "/xyz/openbmc_project/certs",
    "/xyz/openbmc_project/console",
    "/xyz/openbmc_project/control",
    "/xyz/openbmc_project/dump",
    "/xyz/openbmc_project/events",
    "/xyz/openbmc_project/inventory",
    "/xyz/openbmc_project/ipmi",
    "/xyz/openbmc_project/led",
    "/xyz/openbmc_project/logging",
    "/xyz/openbmc_project/network",
    "/xyz/openbmc_project/object_mapper",
    "/xyz/openbmc_project/sensors",
    "/xyz/openbmc_project/software",
    "/xyz/openbmc_project/state",
    "/xyz/openbmc_project/time",
    "/xyz/openbmc_project/user"
  ],
  "message": "200 OK",
  "status": "ok"
}

```

This shows that there are 19 children of the `openbmc_project/` object: dump, software, control, network, logging, etc. This can be used with the base REST URL (ie., `http://${bmc}/`), to discover all objects in the hierarchy.

- Performing the same query with `/list` will list the child objects *recursively*.

```

curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/network/list
{
  "data": [
    "/xyz/openbmc_project/network/config",

```

```

"/xyz/openbmc_project/network/config/dhcp",
"/xyz/openbmc_project/network/eth0",
"/xyz/openbmc_project/network/eth0/ipv4",
"/xyz/openbmc_project/network/eth0/ipv4/3b9faa36",
"/xyz/openbmc_project/network/eth0/ipv6",
"/xyz/openbmc_project/network/eth0/ipv6/ff81b6d6",
"/xyz/openbmc_project/network/eth1",
"/xyz/openbmc_project/network/eth1/ipv4",
"/xyz/openbmc_project/network/eth1/ipv4/3b9faa36",
"/xyz/openbmc_project/network/eth1/ipv4/66e63348",
"/xyz/openbmc_project/network/eth1/ipv6",
"/xyz/openbmc_project/network/eth1/ipv6/ff81b6d6",
"/xyz/openbmc_project/network/host0",
"/xyz/openbmc_project/network/host0/intf",
"/xyz/openbmc_project/network/host0/intf/addr",
"/xyz/openbmc_project/network/sit0",
"/xyz/openbmc_project/network/snmp",
"/xyz/openbmc_project/network/snmp/manager"
],
"message": "200 OK",
"status": "ok"
}

```

- Adding /enumerate instead of /list will also include the attributes of the listed objects.

```

curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/time/enumerate
{
  "data": {
    "/xyz/openbmc_project/time/bmc": {
      "Elapsed": 1563209492098739
    },
    "/xyz/openbmc_project/time/host": {
      "Elapsed": 1563209492101678
    },
    "/xyz/openbmc_project/time/owner": {
      "TimeOwner": "xyz.openbmc_project.Time.Owner.Owners.BMC"
    },
    "/xyz/openbmc_project/time/sync_method": {
      "TimeSyncMethod": "xyz.openbmc_project.Time.Synchronization.Method.NTP"
    }
  },
  "message": "200 OK",
  "status": "ok"
}

```

1.3 HTTP PUT operations

PUT operations are for updating an existing resource (an object or property), or for creating a new resource when the client already knows where to put it. These require a json formatted payload. To get an example of what that looks like:

```

curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/state/host0 > host.json
cat host.json

```

```
{
  "data": {
    "AttemptsLeft": 3,
    "BootProgress": "xyz.openbmc_project.State.Boot.Progress.ProgressStages.Unspecified",
    "CurrentHostState": "xyz.openbmc_project.State.Host.HostState.Off",
    "OperatingSystemState": "xyz.openbmc_project.State.OperatingSystem.Status.OSStatus.Inactive",
    "RequestedHostTransition": "xyz.openbmc_project.State.Host.Transition.Off"
  },
  "message": "200 OK",
  "status": "ok"
}
```

or

```
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/state/host0/attr/RequestedHostTransition
cat requested_host.json
```

```
{
  "data": "xyz.openbmc_project.State.Host.Transition.Off",
  "message": "200 OK",
  "status": "ok"
}
```

When turning around and sending these as requests, delete the message and status properties.

To make curl use the correct content type header use the -H option to specify that we're sending JSON data:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT -d <json> <url>
```

A PUT operation on an object requires a complete object. For partial updates there is PATCH but that is not implemented yet. As a workaround individual attributes are PUTable.

For example, make changes to the requested_host.json file and do a PUT (upload):

```
$ cat requested_host.json
{"data": "xyz.openbmc_project.State.Host.Transition.Off"}
$ curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT -T requested_host.json https://
```

Alternatively specify the json inline with -d:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT -d '{"data": "xyz.openbmc_project.State.Host.Transition.Off"}' https://
```

When using '-d' just remember that json requires quoting.

1.4 HTTP POST operations

POST operations are for calling methods, but also for creating new resources when the client doesn't know where to put it. OpenBMC does not support creating new resources via REST so any attempt to create a new resource will result in a HTTP 403 (Forbidden). These also require a json formatted payload. To delete logging entries:

```
curl -k -H "X-Auth-Token: $token" -H 'Content-Type: application/json' -X POST -d '{"data":[]}' https://${bmc}/xyz/openbmc_project/logging/entry
```

To invoke a method without parameters (Factory Reset of BMC and Host):

```
curl -k -H "X-Auth-Token: $token" -H 'Content-Type: application/json' -X POST -d '{"data":[]}' https://${bmc}/xyz/openbmc_project/factory-reset
```

1.5 HTTP DELETE operations

DELETE operations are for removing instances. Only D-Bus objects (instances) can be removed. If the underlying D-Bus object implements the `xyz.openbmc_project.Object.Delete` interface the REST server will call it. If `xyz.openbmc_project.Object.Delete` is not implemented, the REST server will return a HTTP 403 (Forbidden) error.

For example, to delete a event record:

Display logging entries:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X GET https://${bmc}/xyz/openbmc_projects/1
```

Then delete the event record with ID 1:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X DELETE https://${bmc}/xyz/openbmc_projects/1
```

1.6 Uploading images

It is possible to upload software upgrade images (for example to upgrade the BMC or host software) via REST. The content-type should be set to “application/octet-stream”.

For example, to upload an image:(the `<file_to_upload>` must be a tar ball)

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/octet-stream" -X POST -T <file_to_upload> https://${bmc}/upload
```

In above example, the filename on the BMC will be chosen by the REST server.

It is possible for the user to choose the uploaded file’s remote name:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/octet-stream" -X PUT -T foo https://${bmc}/upload/foo
```

In above example, the file foo will be saved with the name bar on the BMC.

The operation will either return the version id (hash) of the uploaded file on success:

```
{
  "data": "afb92384",
  "message": "200 OK",
  "status": "ok"
}
```

or an error message

```
{
  "data": {
    "description": "Version already exists or failed to be extracted"
  },
  "message": "400 Bad Request",
  "status": "error"
}
```

For more details on uploading and updating software, see: <https://github.com/openbmc/docs/tree/master/architecture/code-update>

1.7 Event subscription protocol

It is possible to subscribe to events, of interest, occurring on the BMC. The implementation on the BMC uses Web-Sockets for this purpose, so that clients don’t have to employ polling. Instead, the rest server on the BMC can

push data to clients over a websocket. The BMC can push out information pertaining to D-Bus InterfacesAdded and PropertiesChanged signals.

Following is a description of the event subscription protocol, with example JS code snippets denoting client-side code.

- a. The client needs to have logged on to the BMC.
- b. The client needs to open a secure websocket with the URL <BMC IP>/subscribe.

```
var ws = new WebSocket("wss://<BMC IP>/subscribe");
```

- c. The client needs to send, over the websocket, a JSON dictionary, comprising of key-value pairs. This dictionary serves as the “events filter”. All the keys are optional, so the dictionary can be empty if no filtering is desired. The filters represented by each of the key-value pairs are ORed.

One of the supported keys is “paths”. The corresponding value is an array of D-Bus paths. The InterfacesAdded and PropertiesChanged D-Bus signals emanating from any of these path(s) alone, and not from any other paths, will be included in the event message going out of the BMC.

The other supported key is “interfaces”. The corresponding value is an array of D-Bus interfaces. The InterfacesAdded and PropertiesChanged D-Bus signal messages comprising of any of these interfaces will be included in the event message going out of the BMC.

All of the following are valid:

```
var data = JSON.stringify({
  paths: ["/xyz/openbmc_project/logging", "/xyz/openbmc_project/sensors"],
  interfaces: [
    "xyz.openbmc_project.Logging.Entry",
    "xyz.openbmc_project.Sensor.Value",
  ],
});
ws.onopen = function () {
  ws.send(data);
};

var data = JSON.stringify({
  paths: ["/xyz/openbmc_project/logging", "/xyz/openbmc_project/sensors"],
});
ws.onopen = function () {
  ws.send(data);
};

var data = JSON.stringify({
  interfaces: [
    "xyz.openbmc_project.Logging.Entry",
    "xyz.openbmc_project.Sensor.Value",
  ],
});
ws.onopen = function () {
  ws.send(data);
};

var data = JSON.stringify({});
ws.onopen = function () {
  ws.send(data);
};
```

```
};
```

- d. The rest server on the BMC will respond over the websocket when a D-Bus event occurs, considering the client supplied filters. The rest servers notifies about InterfacesAdded and PropertiesChanged events. The response is a JSON dictionary as follows :

InterfacesAdded

```
"event": InterfacesAdded
```

```
"path": <string : new D-Bus path that was created>
```

```
"interfaces": <dict : a dictionary of interfaces> (similar to org.freedesktop.DBus.ObjectManager.InterfacesAdded)
```

PropertiesChanged

```
"event": PropertiesChanged
```

```
"path": <string : D-Bus path whose property changed>
```

```
"interface": <string : D-Bus interface to which the changed property belongs>
```

```
"properties": <dict : a dictionary of properties> (similar to org.freedesktop.DBus.Properties.PropertiesChanged)
```

2 Host Management with OpenBMC

This document describes the host-management interfaces of the OpenBMC object structure, accessible over REST.

Note: Authentication

See the details on authentication at [REST-cheatsheet](#).

This document uses token based authentication method:

```
export bmc=xx.xx.xx.xx
```

```
export token=`curl -k -H "Content-Type: application/json" -X POST https://${bmc}/login -d '{"username" : "root", "password" : "root"}'`
```

```
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/...
```

2.1 Inventory

The system inventory structure is under the /xyz/openbmc_project/inventory hierarchy.

In OpenBMC the inventory is represented as a path which is hierarchical to the physical system topology. Items in the inventory are referred to as inventory items and are not necessarily FRUs (field-replaceable units). If the system contains one chassis, a motherboard, and a CPU on the motherboard, then the path to that inventory item would be:

```
inventory/system/chassis0/motherboard0/cpu0
```

The properties associated with an inventory item are specific to that item. Some common properties are:

- Version: A code version associated with this item.
- Present: Indicates whether this item is present in the system (True/False).
- Functional: Indicates whether this item is functioning in the system (True/False).

The usual list and enumerate REST queries allow the system inventory structure to be accessed. For example, to enumerate all inventory items and their properties:

```
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/inventory/enumerate
```

To list the properties of one item:

```
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/inventory/system/chassis/motherboard
```

2.2 Sensors

The system sensor structure is under the `/xyz/openbmc_project/sensors` hierarchy.

This interface allows monitoring of system attributes like temperature or altitude, and are represented similar to the inventory, by object paths under the top-level sensors object name. The path categorizes the sensor and shows what the sensor represents, but does not necessarily represent the physical topology of the system.

For example, all temperature sensors are under `sensors/temperature`. CPU temperature sensors would be `sensors/temperature/cp`

These are some common properties:

- Value: Current value of the sensor
- Unit: Unit of the value and “Critical” and “Warning” values
- Scale: The scale of the value and “Critical” and “Warning” values
- CriticalHigh & CriticalLow: Sensor device upper/lower critical threshold bound
- CriticalAlarmHigh & CriticalAlarmLow: True if the sensor has exceeded the critical threshold bound
- WarningHigh & WarningLow: Sensor device upper/lower warning threshold bound
- WarningAlarmHigh & WarningAlarmLow: True if the sensor has exceeded the warning threshold bound

A temperature sensor might look like:

```
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/sensors/temperature/ocp_zone
{
  "data": {
    "CriticalAlarmHigh": false,
    "CriticalAlarmLow": false,
    "CriticalHigh": 65000,
    "CriticalLow": 0,
    "Functional": true,
    "MaxValue": 0,
    "MinValue": 0,
    "Scale": -3,
    "Unit": "xyz.openbmc_project.Sensor.Value.Unit.DegreesC",
    "Value": 34625,
    "WarningAlarmHigh": false,
    "WarningAlarmLow": false,
    "WarningHigh": 63000,
    "WarningLow": 0
  },
  "message": "200 OK",
  "status": "ok"
}
```

Note the value of this sensor is 34.625C ($34625 * 10^{-3}$).

Unlike IPMI, there are no “functional” sensors in OpenBMC; functional states are represented in the inventory.

To enumerate all sensors in the system:

```
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/sensors/enumerate
```

List properties of one inventory item:

```
curl -k -H "X-Auth-Token: $token" https://${bmc}/xyz/openbmc_project/sensors/temperature/outlet
```


2.3 Event Logs

The event log structure is under the `/xyz/openbmc_project/logging/entry` hierarchy. Each event is a separate object under this structure, referenced by number.

BMC and host firmware on POWER-based servers can report event logs to the BMC. Typically, these event logs are reported in cases where host firmware cannot start the OS, or cannot reliably log to the OS.

The properties associated with an event log are as follows:

- **Message:** The type of event log (e.g. `"xyz.openbmc_project.Inventory.Error.NotPresent"`).
- **Resolved :** Indicates whether the event has been resolved.
- **Severity:** The level of problem (`"Info"`, `"Error"`, etc.).
- **Timestamp:** The date of the event log in epoch time.
- **Associations:** A URI to the failing inventory part.

To list all reported event logs:

```
curl -k -H "X-Auth-Token: $token" https://{bmc}/xyz/openbmc_project/logging/entry
{
  "data": [
    "/xyz/openbmc_project/logging/entry/3",
    "/xyz/openbmc_project/logging/entry/2",
    "/xyz/openbmc_project/logging/entry/1",
    "/xyz/openbmc_project/logging/entry/7",
    "/xyz/openbmc_project/logging/entry/6",
    "/xyz/openbmc_project/logging/entry/5",
    "/xyz/openbmc_project/logging/entry/4"
  ],
  "message": "200 OK",
  "status": "ok"
}
```

To read a specific event log:

```
curl -k -H "X-Auth-Token: $token" https://{bmc}/xyz/openbmc_project/logging/entry/1
{
  "data": {
    "AdditionalData": [
      "_PID=183"
    ],
    "Id": 1,
    "Message": "xyz.openbmc_project.Common.Error.InternalFailure",
    "Purpose": "xyz.openbmc_project.Software.Version.VersionPurpose.BMC",
    "Resolved": false,
    "Severity": "xyz.openbmc_project.Logging.Entry.Level.Error",
    "Timestamp": 1563191362822,
    "Version": "2.8.0-dev-132-gd1c1b74-dirty",
    "associations": []
  },
  "message": "200 OK",
  "status": "ok"
}
```

To delete an event log (log 1 in this example), call the Delete method on the event:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X POST -d '{"data" : []}' https://$bmc
```

To clear all event logs, call the top-level DeleteAll method:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X POST -d '{"data" : []}' https://$bmc
```

2.4 Host Boot Options

With OpenBMC, the Host boot options are stored as D-Bus properties under the control/host0/boot path. Properties include [BootMode](#), [BootSource](#) and if the host is based on x86 CPU also [BootType](#).

- Set boot mode:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT https://$bmc/xyz/openbmc_project
```

- Set boot source:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT https://$bmc/xyz/openbmc_project
```

- Set boot type (valid only if host is based on the x86 CPU):

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT https://$bmc/xyz/openbmc_project
```

Also there are boolean [Enable](#) properties that control if the boot source override is persistent or one-time, and if the override is enabled or not.

- Set boot override one-time flag:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT https://$bmc/xyz/openbmc_project
```

- Enable boot override:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT https://$bmc/xyz/openbmc_project
```

2.5 Host State Control

The host can be controlled through the host object. The object implements a number of actions including power on and power off. These correspond to the IPMI power on and power off commands.

Assuming you have logged in, the following will power on the host:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -d '{"data": "xyz.openbmc_project.State"
```

To power off the host:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -d '{"data": "xyz.openbmc_project.State"
```

To issue a hard power off (accomplished by powering off the chassis):

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT -d '{"data": "xyz.openbmc_project"
```

To reboot the host:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X PUT -d '{"data": "xyz.openbmc_project"
```

More information about Host State Management can be found here: https://github.com/openbmc/phosphor-dbus-interfaces/tree/master/yaml/xyz/openbmc_project/State

2.6 Host Clear GARD

On OpenPOWER systems, the host maintains a record of bad or non-working components on the GARD partition. This record is referenced by the host on subsequent boots to determine which parts should be ignored.

The BMC implements a function that simply clears this partition. This function can be called as follows:

- Method 1: From the BMC command line:

```
busctl call org.open_power.Software.Host.Updater \
  /org/open_power/control/gard \
  xyz.openbmc_project.Common.FactoryReset Reset
```

- Method 2: Using the REST API:

```
curl -k -H "X-Auth-Token: $token" -H "Content-Type: application/json" -X POST -d '{"data":[]}' https://${bmc}
```

Implementation: <https://github.com/openbmc/openpower-pnor-code-mgmt>

2.7 Host Watchdog

The host watchdog service is responsible for ensuring the host starts and boots within a reasonable time. On host start, the watchdog is started and it is expected that the host will ping the watchdog via the inband interface periodically as it boots. If the host fails to ping the watchdog within the timeout then the host watchdog will start a systemd target to go to the quiesce target. System settings will then determine the recovery behavior from that state, for example, attempting to reboot the system.

The host watchdog utilizes the generic [phosphor-watchdog](#) repository. The host watchdog service provides 2 files as configuration options into phosphor-watchdog:

```
/lib/systemd/system/phosphor-watchdog@poweron.service.d/poweron.conf
/etc/default/obmc/watchdog/poweron
```

poweron.conf contains the “Conflicts” relationships to ensure the watchdog service is stopped at the correct times. poweron contains the required information for phosphor-watchdog (more information on these can be found in the [phosphor-watchdog](#) repository).

The 2 service files involved with the host watchdog are:

```
phosphor-watchdog@poweron.service
obmc-enable-host-watchdog@0.service
```

phosphor-watchdog@poweron starts the host watchdog service and obmc-enable-host-watchdog starts the watchdog timer. Both are run as a part of the obmc-host-startmin@.target. Service dependencies ensure the service is started before the enable is called.

The default watchdog timeout can be found within the [dbus interface specification](#) (Interval property).

The host controls the watchdog timeout and enable/disable once it starts.

3 OpenBMC host console support

This document describes how to connect to the host UART console from an OpenBMC management system.

The console infrastructure allows multiple shared connections to a single host UART. UART data from the host is output to all connections, and input from any connection is sent to the host.

3.1 Remote console connections

To connect to an OpenBMC console session remotely, just ssh to your BMC on port 2200. Use the same login credentials you would for a normal ssh session:

```
ssh -p 2200 [user]@[bmc-hostname]
```

3.2 Local console connections

If you're already logged into an OpenBMC machine, you can start a console session directly, using:

```
obmc-console-client
```

To exit from a console, type:

```
return ~ .
```

Note that if you're on an ssh connection, you'll need to 'escape' the ~ character, by entering it twice:

```
return ~ ~ .
```

This is because obmc-console-client is an ssh session, and a double ~ is required to escape the "inner" (obmc-console-client) ssh session.

3.3 Logging

Console logs are kept in: `/var/log/obmc-console.log`

This log is limited in size, and will wrap after hitting that limit (currently set at 16kB).

4 OpenBMC Code Update

Two BMC Code layouts are available:

- Static, non-UBI layout
- UBI layout - enabled via `obmc-ubi-fs` distro feature

This document describes the code update that supports both layouts.

4.1 Steps to Update

The following are the steps to update the BMC.

1. Get a BMC image tar: After building OpenBMC, you will end up with a set of image files in `tmp/deploy/images/<platform>/`.
 - The UBI layout image is `obmc-phosphor-image-<platform>-<timestamp>.ubi.mtd.tar`
 - The static layout image is `obmc-phosphor-image-<platform>-<timestamp>.static.mtd.tar`

The BMC tar image contains 5 files: u-boot, kernel, ro, and rw partitions and the MANIFEST file, which contains information about the image such as the image purpose, version, KeyType (Key type used for signature), HashType (SHA type used for key generation) and MachineName (name of machine used while building image, and this will be used for validation of image build). A MANIFEST file might look like

```
purpose=xyz.openbmc_project.Software.Version.VersionPurpose.BMC
version=2.7.0-dev
KeyType=OpenBMC
HashType=RSA-SHA256
MachineName=tiogapass
```

2. Transfer the generated BMC image to the BMC via one of the following methods:
 - Method 1: Via Redfish Upload: <https://github.com/openbmc/docs/blob/master/REDFISH-cheatsheet.md#firmware-update>. If using this method skip ahead to step 5!
 - Method 2: Via scp: Copy the generated BMC image to the /tmp/images/ directory on the BMC.
 - Method 3: Via REST Upload: <https://github.com/openbmc/docs/blob/master/rest-api.md#uploading-images>
 - Method 4: Via TFTP: Perform a POST request to call the DownloadViaTFTP method of /xyz/openbmc_project/software.

Methods 3 and 4 require additional options in bmcweb to be enabled.

3. Note the version id generated for that image file. The version id is a hash value of 8 hexadecimal numbers, generated by SHA-512 hashing the version string contained in the image and taking the first 8 characters. Get the version id via one of the following methods:

- Method 1: From the BMC command line, note the most recent directory name created under /tmp/images/, in this example it'd be 2a1022fe:

```
# ls -l /tmp/images/
total 0
drwx----- 2 root root 80 Aug 22 07:54 2a1022fe
drwx----- 2 root root 80 Aug 22 07:53 488449a2
```

- Method 2: This method *only* works if there are no Ready images at the start of transferring the image. Using the REST API, note the object that has its Activation property set to Ready, in this example it'd be 2a1022fe:

```
$ curl -b cjar -k https://${bmc}/xyz/openbmc_project/software/enumerate
{
  "data": {
    "/xyz/openbmc_project/software/2a1022fe": {
      "Activation": "xyz.openbmc_project.Software.Activation.Activations.Ready",
```

- Method 3: Calculate the version id beforehand from the image with:

```
tar xfo <BMC tar image> MANIFEST | sed -ne '/version=/ {s/version=//;p}' | head -n1 | tr -d '\n' | sha
```

4. To initiate the update, set the RequestedActivation property of the desired image to Active, substitute <id> with the hash value noted on the previous step, this will write the contents of the image to the BMC chip via one of the following methods:

- Method 1: From the BMC command line:

```
busctl set-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.Activation RequestedActivation s \
  xyz.openbmc_project.Software.Activation.RequestedActivations.Active
```

- Method 2: Using the REST API:

```
curl -b cjar -k -H "Content-Type: application/json" -X PUT \
  -d '{"data":
    "xyz.openbmc_project.Software.Activation.RequestedActivations.Active"}' \
  https://${bmc}/xyz/openbmc_project/software/<id>/attr/RequestedActivation
```

5. (Optional) Check the flash progress. This interface is only available during the activation progress and is not present once the activation is completed via one of the following:

- Method 1: From Redfish: A task is returned from the Redfish upload. The task can be used to monitor the progress.

```
curl -k https://${bmc}/redfish/v1/TaskService/Tasks/0
```

- Method 2: From the BMC command line:

```
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.ActivationProgress Progress
```

- Method 3: Using the REST API:

```
curl -b cjar -k https://${bmc}/xyz/openbmc_project/software/<id>/attr/Progress
```

6. Check that the activation is complete by verifying the “Activation” property is set to “Active” via one of the following methods:

- Method 1: From Redfish: Check the task returned from the Redfish upload.

```
curl -k https://${bmc}/redfish/v1/TaskService/Tasks/0
```

- Method 2: From the BMC command line:

```
busctl get-property xyz.openbmc_project.Software.BMC.Updater \
  /xyz/openbmc_project/software/<id> \
  xyz.openbmc_project.Software.Activation Activation
```

- Method 3: Using the REST API:

```
curl -b cjar -k https://${bmc}/xyz/openbmc_project/software/<id>
```

7. Reboot the BMC for the image to take effect.

- Method 1: From Redfish: If ApplyTime was set to “Immediate”, the BMC will automatically reboot: <https://github.com/openbmc/docs/blob/master/REDFISH-cheatsheet.md#firmware-applytime>. To reboot the BMC manually see: <https://github.com/openbmc/docs/blob/master/REDFISH-cheatsheet.md#bmc-reboot>.

- Method 2: From the BMC command line:

```
reboot
```

- Method 3: Using the REST API:

```
curl -c cjar -b cjar -k -H "Content-Type: application/json" -X PUT \
  -d '{"data": "xyz.openbmc_project.State.BMC.Transition.Reboot"}' \
  https://${bmc}/xyz/openbmc_project/state/bmc0/attr/RequestedBMCTransition
```

4.2 Associations

In addition to all software images, several associations are listed at /xyz/openbmc_project/software/:

```
curl -c cjar -b cjar -k -H "Content-Type: application/json" -X GET \
  https://${bmc}/xyz/openbmc_project/software/
{
  "data": [
    "/xyz/openbmc_project/software/46e65782",
    "/xyz/openbmc_project/software/493a00ad",
    "/xyz/openbmc_project/software/88c153b1",
    "/xyz/openbmc_project/software/active",
    "/xyz/openbmc_project/software/functional"
  ],
  "message": "200 OK",
}
```

```
"status": "ok"
}
```

1. A “functional” association to the “running” BMC and host images

There is only one functional association per BMC and one functional association per host. The functional/running BMC image is the BMC image with the lowest priority when rebooting the BMC. The functional image does not update until the BMC is rebooted. The functional host image behaves the same way except that it updates on a power on or reboot of the host.

```
curl -c cjar -b cjar -k -H "Content-Type: application/json" -X GET \
  https://${bmc}/xyz/openbmc_project/software/functional
{
  "data": {
    "endpoints": [
      "/xyz/openbmc_project/software/46e65782",
      "/xyz/openbmc_project/software/493a00ad"
    ]
  },
  "message": "200 OK",
  "status": "ok"
}
```

2. An “active” association to the active BMC and host images

Note: Several BMC images might be active, this is true for the host images as well.

```
curl -c cjar -b cjar -k -H "Content-Type: application/json" -X GET \
  https://${bmc}/xyz/openbmc_project/software/active
{
  "data": {
    "endpoints": [
      "/xyz/openbmc_project/software/46e65782",
      "/xyz/openbmc_project/software/493a00ad",
      "/xyz/openbmc_project/software/88c153b1"
    ]
  },
  "message": "200 OK",
  "status": "ok"
}
```

3. An “updateable” association to the programmable components

This is used for identifying firmware components which are programmable via BMC OOB interfaces like Redfish/IPMI. All updateable firmware components must expose the updateable association so that upper applications like Redfish/IPMI will know about updateable firmwares.

To know the updateable software components:

```
busctl call xyz.openbmc_project.ObjectMapper \
  /xyz/openbmc_project/software/updatable org.freedesktop.DBus.Properties \
  Get ss xyz.openbmc_project.Association endpoints
v as 1 "/xyz/openbmc_project/software/1201fc36"
```

Redfish interface uses ‘updateable’ association in SoftwareInventory schema.

4. An additional association is located at /xyz/openbmc_project/software/<id>/inventory for “associating” a software image with an inventory item.

```
curl -c cjar -b cjar -k -H "Content-Type: application/json" -X GET \
https://${bmc}/xyz/openbmc_project/software/493a00ad/inventory
{
  "data": {
    "endpoints": [
      "/xyz/openbmc_project/inventory/system/chassis/motherboard/boxelder/bmc"
    ]
  },
  "message": "200 OK",
  "status": "ok"
}
```

To get all software images associated with an inventory item:

```
curl -c cjar -b cjar -k -H "Content-Type: application/json" -X GET \
https://${bmc}/xyz/openbmc_project/inventory/system/chassis/activation
{
  "data": {
    "endpoints": [
      "/xyz/openbmc_project/software/46e65782"
    ]
  },
  "message": "200 OK",
  "status": "ok"
}
```

4.3 MANIFEST File

A file named “MANIFEST” must be included in any image tar uploaded, downloaded via TFTP, or copied to the BMC.

The MANIFEST file format must be key=value (e.g. version=v1.99.10). It should include the following fields:

- version - The version of the image
- purpose - The image’s purpose (e.g. xyz.openbmc_project.Software.Version.VersionPurpose.BMC or xyz.openbmc_project.Software.Version.VersionPurpose.ACCEPTED). Accepted purpose values can be found at [Version interface](#) under “VersionPurpose” values.
- MachineName - The name of machine (platform) for which this image is built for. This value will be compared against OPENBMC_TARGET_MACHINE value defined in os-release file of running image. Image will not be upgraded if this check fails. For backward compatibility this check skips failure if MachineName is not defined for current released images but it will be made mandatory field from 2.9 onward releases.

Other optional fields are:

- extended_version - A more detailed version, which could include versions of different components in the image.

4.4 Deleting an Image

To delete an image:

```
curl -c cjar -b cjar -k -H "Content-Type: application/json" \
-X POST https://${bmc}/xyz/openbmc_project/software/<$id>/action/delete \
-d "{\"data\": [] }"
```


Note: The image must be non-functional (“non-running”).

To delete all non-functional images, whether BMC or host images:

```
curl -c cjar -b cjar -k -H "Content-Type: application/json" \
  -X POST https://${bmc}/xyz/openbmc_project/software/action/deleteAll \
  -d '{"data": []}'
```

4.5 Software Field Mode

Field mode is meant for systems shipped from manufacturing to a customer. Field mode offers a way to provide security and ensure incorrect patches don't get loaded on the system by accident. The software implementation of the field mode interface disables patching of the BMC by not mounting `/usr/local`, which in turn disables host patching at `/usr/local/share/pnor/`. Enabling field mode is intended to be a one-way operation which means that once enabled, there is no REST API provided to disable it.

Field mode can be enabled by running the following command:

```
curl -b cjar -k -H 'Content-Type: application/json' -X PUT -d '{"data":1}' \
  https://${bmc}/xyz/openbmc_project/software/attr/FieldModeEnabled
```

Although field mode is meant to be a one-way operation, it can be disabled by a user with admin privileges by running the following commands on the BMC:

```
fw_setenv fieldmode
```

```
systemctl unmask usr-local.mount
```

```
reboot
```

More information on field mode can be found here: https://github.com/openbmc/phosphor-dbus-interfaces/blob/master/yaml/xyz/openbmc_project/Control/FieldMode.interface.yaml

4.6 Software Factory Reset

Software factory reset resets the BMC and host firmware to its factory state by clearing out any read/write data. To software factory reset run the following command and then reboot the BMC:

```
curl -b cjar -k -H 'Content-Type: application/json' -X POST -d '{"data":[]}' \
  https://${bmc}/xyz/openbmc_project/software/action/Reset
```

The factory reset on the BMC side will clear `/var`, `/home`, and `/etc`. On the host side, the factory reset will clear the read/write volume for each host image on the system, clear the shared preserve host volume, `pnor-prsv`, and clear any host patches located in `/usr/local/share/pnor/`.

The factory reset interface can be found here: https://github.com/openbmc/phosphor-dbus-interfaces/blob/02b39246d45ea029a1652a49cc20eab7723dd63b/xyz/openbmc_project/Common/FactoryReset.interface.yaml

4.7 Image Storage Location

4.7.1 Static layout

When a BMC image is activated, each image-<name> is written to the BMC chip's partitions indicated by the <name>:

- image-u-boot
- image-kernel
- image-rofs
- image-rwfs

4.7.2 UBI layout

When a BMC image is activated (i.e. when “RequestedActivation” is set to “Active”), UBI volumes are created on the BMC chip for the image. The alternate BMC chip can also be used to store images. This is determined by “BMC_RO_MTD”. Using both the alternate BMC chip and the BMC chip allows for multiple BMC images to be stored. By default, only the BMC chip is used. To use both, set “BMC_RO_MTD” to “alt-bmc+bmc”.

4.8 Implementation

More information about the implementation of the code update can be found at https://github.com/openbmc/phosphor-dbus-interfaces/blob/master/yaml/xyz/openbmc_project/Software and <https://github.com/openbmc/phosphor-bmc-code-mgmt>